

COMM114: Generative AI -Revision

Guoqiang Zhang

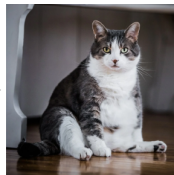
December 2025

Definition of Generative AI (Lect. 1)

Generative artificial intelligence (Generative AI) is a subfield of artificial intelligence that uses **generative models** to produce text, images, videos, or other forms of data. These models learn the underlying **patterns and structures** of their **training data** and use them to produce **new data** based on the input, which often comes in the form of natural language prompts. (from wikipedia).

Prompt:

“a picture of cat”



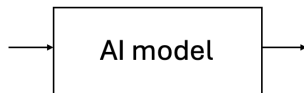
New sample

Definition of Generative AI (cont.)

Generative artificial intelligence (Generative AI) is a subfield of artificial intelligence that uses **generative models** to produce text, images, videos, or other forms of data. These models learn the underlying **patterns and structures** of their **training data** and use them to produce **new data** based on the input, which often comes in the form of natural language prompts. (from wikipedia).

Prompt:

“A ballet dancer twirls
on the surface of a still
lake at sunset”



[a video sample](#)

Remark: The link for the video sample is [here](#).

From Generation to Sampling (cont.)

- An AI model is in fact a probabilistic generative model (PGM).
- **Definition of PGM:** A probabilistic generative model (PGM) is a probability distribution over a random variable $\mathbf{X}$ which attempts to learn from a set of observed data $\{\mathbf{x}_i \sim p_{data}(\mathbf{x})\}_{i=1}^n$ with some probability density $p_{\theta}(\mathbf{x})$, which is parameterized by θ .
- Generation of new samples from an AI model is in fact sampling from a PGM $p_{\theta}(\mathbf{x})$.

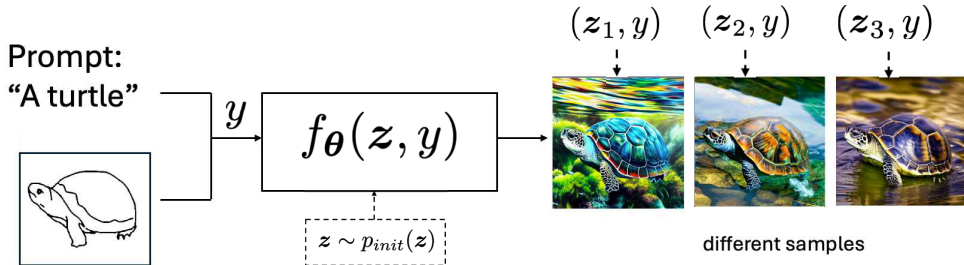
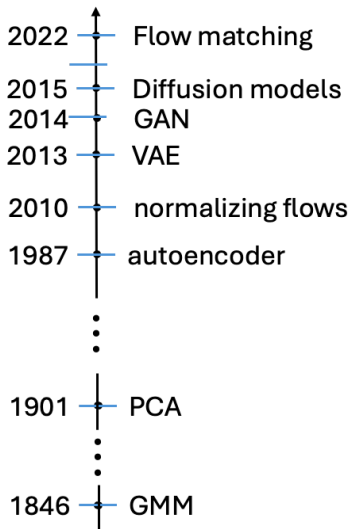


Figure: Demonstration of image generation for the 2nd class of PGMs

Generative AI History (Lect. 1)

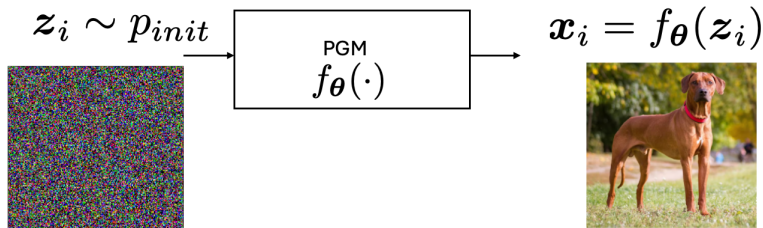
Different learning objectives and approaches lead to different PGMs (or generative AI models), e.g., GANs, VAEs, NFs etc.



Generative AI history (Lect. 1)

Classification of PGMs

- Input data: $\{\mathbf{x}_i\}_{i=1}^n$
- Some PGMs **directly** learn $p_{\theta}(\mathbf{x})$ from the dataset $\{\mathbf{x}_i\}_{i=1}^n$.
 - Gaussian mixture model (GMM), normalizing flows (NFs)
- Other PGMs **implicitly** learn $p_{\theta}(\mathbf{x})$ from the dataset $\{\mathbf{x}_i\}_{i=1}^n$. In particular, they learn to convert samples from an initial distribution (e.g., Gaussian distribution) into samples from the data distribution.



- In the diagram above, the random variable $\mathbf{z}$ is referred to as the latent variable.

Probability theory (Lect. 2)

- Continuous random variables
- Discrete random variables
- Chain rule of probability
- Bayesian formula
- Kullback-Leibler (KL) divergence

Kullback-Leibler divergence (Lect. 2)

The Kullback-Leibler (KL) divergence measures how much a model probability distribution Q is different from a true probability distribution P :

$$D_{KL}(P||Q) = \int p(\mathbf{x}) \log \left(\frac{p(\mathbf{x})}{q(\mathbf{x})} \right) d\mathbf{x}$$

Properties of KL divergence

- Not symmetric.
 - $D_{KL}(P||Q) \neq D_{KL}(Q||P)$
- $D_{KL}(P||Q)$ is always ≥ 0 .
- $D_{KL}(P||Q)$ is equal to 0 if and only if $P = Q$. That is, only if the two probability distribution are identical, $D_{KL}(P||Q) = 0$.

Maximum Likelihood Estimation (Lect. 3)

Definition

Maximum likelihood estimation (MLE) is a method of estimating the parameters of an assumed probability distribution, given some observed data. This is achieved by maximizing a likelihood function so that, under the assumed statistical model, the observed data is most probable. (from wiki)

Problem Formulation:

- Input data set $\{\mathbf{x}_i \in \mathbb{R}^d | i = 1, 2, \dots, n\}$
- Select a parametric probability model $p_{\theta}(\cdot)$
- Fitting the data by maximizing the following likelihood function

$$\begin{aligned}\theta_{MLE}^* &= \arg \max_{\theta} \prod_{i=1}^n p_{\theta}(\mathbf{x}_i) && \text{[product of likelihood]} \\ &= \arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n \log p_{\theta}(\mathbf{x}_i) && \text{[summation of log likelihood]}\end{aligned}$$

Gaussian Mixture Model (Lect. 3)

- A Gaussian Mixture Model (GMM) is a probabilistic generative model (PGM) that assumes a dataset's underlying distribution is a weighted sum of multiple Gaussian (normal) distributions.
- Its PDF $p_{\theta}(\mathbf{x})$ takes the form of

$$p_{\theta}(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

- K is the number of components in GMM.
- π_k is the weight for the k th component, which tells us how much that particular Gaussian contributes to the overall model. These weights must satisfy

$$\{\pi_k \geq 0\}_{k=1}^K, \text{ and } \sum_{k=1}^K \pi_k = 1$$

- $\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ is the Gaussian distribution for the k th component.
- The parameter θ includes $\theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$.

Gaussian Mixture Model (Lect. 3)

- GMM has been studied extensively in the literature.
- Trained either via ML or a variational bound on likelihood
- Sampling and evaluating $p_{\theta}(\mathbf{x})$ is straightforward.
- Performance scales poorly with dimensionality.

VAEs (Lect. 4-6)

- A VAE learns stochastic mappings between an observed $\mathbf{x}$ -space and a latent $\mathbf{z}$ space.
- The generative model learns a joint distribution $p_{\theta}(\mathbf{x}, \mathbf{z}) = p_{\theta}(\mathbf{z})p_{\theta}(\mathbf{x}|\mathbf{z})$. The stochastic decoder learns the conditional probability $p_{\theta}(\mathbf{x}|\mathbf{z})$.
- $p_{\theta}(\mathbf{z})$ is the prior distribution of the latent random variable $\mathbf{z}$.
- The stochastic encoder learns an approximation $q_{\varphi}(\mathbf{z}|\mathbf{x})$ of the true but intractable posterior $p_{\theta}(\mathbf{z}|\mathbf{x})$ of the generative model.
-

Remark: $q_{\varphi}(\mathbf{z}|\mathbf{x})$ is referred to as the approximate posterior.

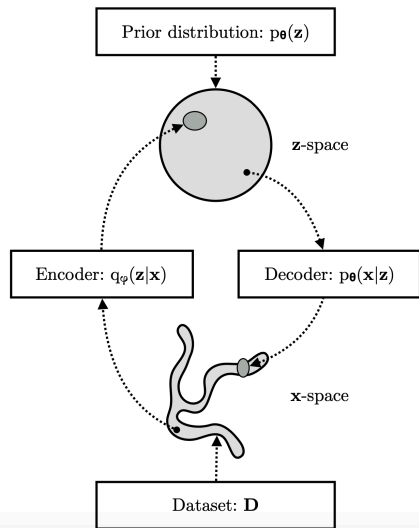
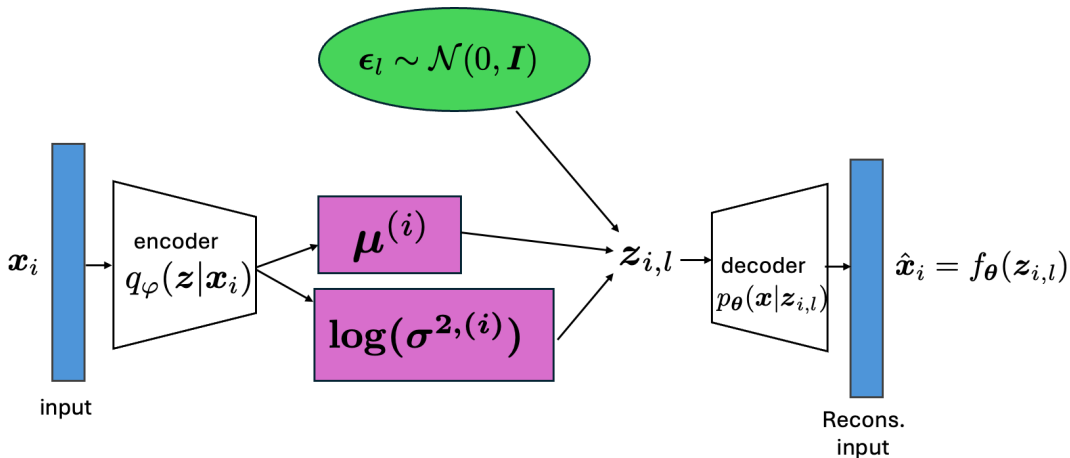


Diagram when training a VAE (Lect. 4-6)



- For each input sample x_i , one can obtain a number of different reconstructions $f_\theta(z_{i,l})$ depending on different Gaussian noises ϵ_l , $l = 1, \dots, L$.

Objective function for training a VAE (Lect. 4-6)

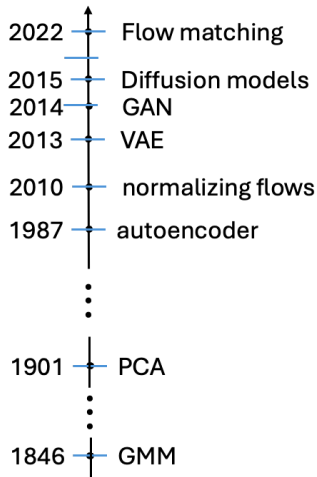
The ELBO function to be minimized is

$$\arg \min_{\theta, \varphi} \frac{1}{n} \sum_{i=1}^n \left[\underbrace{D_{KL}(\mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2, (i)} \mathbf{I}) || \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}))}_{\text{1st term}} + \frac{1}{L} \sum_{l=1}^L \underbrace{(\mathbf{x}_i - f_{\theta}(\mathbf{z}_{i,l}))^2}_{\text{2nd term}} \right] \quad (1)$$

- The 2nd term minimize the reconstruction error between each input sample $\mathbf{x}_i$ and its associated output $f_{\theta}(\mathbf{z}_{i,l})$.
- The first term makes the approximate posterior $\mathcal{N}(\mathbf{z}; \boldsymbol{\mu}^{(i)}, \boldsymbol{\sigma}^{2, (i)} \mathbf{I})$ of the stochastic encoder as close as possible to the prior distribution $\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$.

GANs (Lect. 7-8)

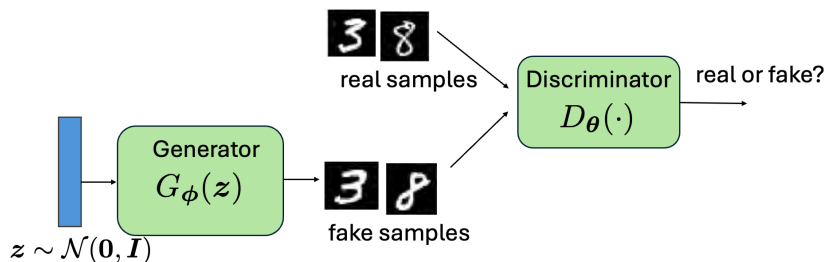
Generative adversarial network (GAN) was invented in 2014 by Ian Goodfellow and his colleagues in June 2014.



GANs (Lect. 7-8)

GAN introduces a discriminator $D_{\theta}(\cdot)$ to help training the generator $G_{\phi}(\cdot)$.

- In the training process, the generator updates its parameter ϕ by attempting to produce more realistic (fake) samples that makes the discriminator thinks that the samples came from the real data distribution.
- The discriminator updates its parameter θ by distinguishing between real and fake samples more accurately.



Remark: $D_{\theta}(\cdot)$ is a scalar function since the discriminator solves a binary (real versus fake samples) **classification problem**.

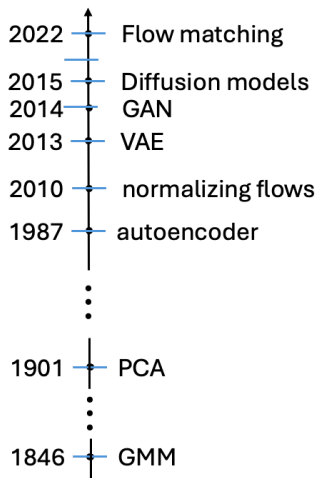
GANs (Lect. 7-8)



Figure: Example of fake images generated by a pre-trained GAN (named styleGAN)

Normalizing Flows (Lect. 10-11)

Normalizing flow (NF) was invented in 2010.



Objective of NF (Lect. 10-11)

- **Input:** data samples $\{\mathbf{x}_i\}_{i=1}^n$.



- **Assumption:** Assume that the data samples are drawn from a certain data distribution $p_{data}(\cdot)$.
- **Objective of NF:** To learn an **invertible and differentiable** function $f_{\theta}(\cdot)$ (e.g., a deep neural network) that maps a Gaussian noise $\mathbf{z} \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$ to a sample $f_{\theta}(\mathbf{z})$ that looks as if it came from the data distribution $p_{data}(\mathbf{x})$.
- Unlike GAN, no discriminator is introduced in NF.
- **The invertibility and differentiability of $f_{\theta}(\cdot)$ makes the learning process much easier.**
- To facilitate invertibility of f_{θ} , the input and output of $f_{\theta}(\cdot)$ are set to the same dimension.

Training via Maximum log-likelihood

Explicit PDF for $x = f_{\theta}(z)$ makes the training process much easier

- Input: data samples $\{x_i\}_{i=1}^n$
- The parameter θ can be learned by maximizing the log-likelihood via

$$\begin{aligned} & \arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n \log(p_X(x_i; \theta)) \\ &= \arg \max_{\theta} \frac{1}{n} \sum_{i=1}^n \left(\log p_Z(f_{\theta}^{-1}(x_i)) + \log |\det Df_{\theta}^{-1}(x_i)| \right), \end{aligned}$$

where the $Df_{\theta}^{-1}(x_i)$ is the Jacobian matrix.

Composition of Flows (Lect. 10-11)

- To facilitate the design of f_{θ} in NF, x is computed as

$$\begin{aligned}x &= f_{\theta}^m \circ \cdots \circ f_{\theta}^1(z) \\&= f_{\theta}^m(f_{\theta}^{m-1}(\cdots(f_{\theta}^1(z)))) \\&= f_{\theta}(z).\end{aligned}$$

- The overall function f_{θ} consists of a sequence of m invertible mappings to obtain x .
- Each mapping $f_{\theta}^j(\cdot)$ can be a relatively simple function.

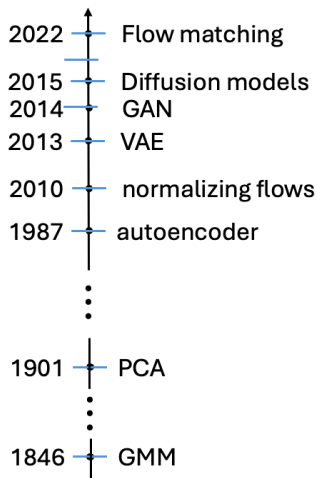
Demonstration of NF (Lect. 10-11)



Picture from "Glow: Generative Flow with Invertible 1×1 Convolutions" by Diederik P. Kingma and Prafulla Dhariwal.

Flow Matching (Lect. 12-13)

Flow matching (FM) was invented in 2022 as a generalization of normalizing flows.



Objective of Flow Matching Models (Lect. 12-13)

- **Input:** data samples $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^n$.



- **Assumption:** Assume $\{\mathbf{x}_i\}_{i=1}^n$ are drawn from the data distribution $p_{data}(\cdot)$.
- **Objective of FM:** To learn a **flow** $\mathbf{X}_t^\theta \in \mathbb{R}^d$ over the time interval $t \in [0, 1]$ such that

$$p_{\mathbf{X}_{t=0}^\theta}(\mathbf{x}_{t=0}) = \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad [\text{PDF of flow at } t=0] \quad (2)$$

$$p_{\mathbf{X}_{t=1}^\theta}(\mathbf{x}_{t=1}) \approx p_{data}(\mathbf{x}) \quad [\text{PDF of flow at } t=1] \quad (3)$$

- For each $t \in [0, 1]$, $\mathbf{X}_t^\theta$ is a d -dimensional continuous random variable.
- θ is the parameter of a deep neural network to be learned.

Velocity field of an FM model (Lect. 12-13)

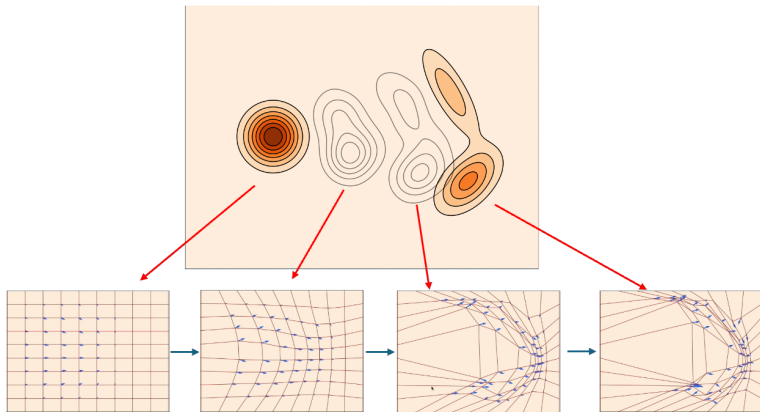
A velocity field is easier to learn:

- **Input:** data samples $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^n$.
- **Assumption:** Assume $\{\mathbf{x}_i\}_{i=1}^n$ are drawn from the data distribution $p_{data}(\cdot)$.
- Instead of learning the flow $\{\mathbf{X}_t^\theta \in \mathbb{R}^d | t \in [0, 1]\}$ directly, we learn a so-called **velocity field** $\mathbf{u}_t^\theta(\mathbf{X}_t^\theta)$ over $t \in [0, 1]$ from the dataset $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^n$.
- The velocity field $\mathbf{u}_t^\theta(\mathbf{X}_t^\theta)$ is related with the flow $\mathbf{X}_t^\theta$ via an ordinary differential equation (ODE):

$$\frac{d}{dt}\mathbf{X}_t^\theta = \mathbf{u}_t^\theta(\mathbf{X}_t^\theta) \quad \text{s. t. } \mathbf{X}_{t=0}^\theta \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad [\text{ODE}]$$

- Once the velocity field $\mathbf{u}_t^\theta(\cdot)$ is obtained, the flow $\{\mathbf{X}_t^\theta | t \in [0, 1]\}$ is the solution of the ODE.

Velocity field of an FM model (cont.)



Dynamics of velocity field over $t \in [0, 1]$

- The grid points represent flow vectors.
- The **blue** arrows denote the velocities of the flow vectors.
- picture from [Tutorial on Flow Matching for Generative Modeling](#)

Training of velocity field

- Let PDF of $\mathbf{X}_0$ be $p_{init} = \mathcal{N}(\mathbf{0}, \mathbf{I})$ (i.e., $p_{\mathbf{X}_0}(\mathbf{x}_0) = p_{init}(\mathbf{x})$)
- Let PDF of $\mathbf{X}_1$ be $p_{data}(\mathbf{x})$ (i.e., $p_{\mathbf{X}_1}(\mathbf{x}_1) = p_{data}(\mathbf{x})$)
- Define $\mathbf{X}_t = \beta_t \mathbf{X}_0 + \alpha_t \mathbf{X}_1$ where α_t and β_t satisfy:
 - α_t and β_t are scalar differentiable functions over $t \in [0, 1]$.
 - When $t = 0$, $\alpha_{t=0} = 0$ and $\beta_{t=0} = 1$.
 - When $t = 1$, $\alpha_{t=1} = 1$ and $\beta_{t=1} = 0$.
- Objective function for $\mathbf{u}_t^\theta(\cdot)$:

$$\begin{aligned} & \arg \min_{\theta} \mathbb{E}_{t, \mathbf{X}_1, \mathbf{X}_0} \left\| \mathbf{u}_t^\theta(\mathbf{X}_t) - \mathbf{u}_t^{\text{target}}(\mathbf{X}_t) \right\|^2 \\ &= \arg \min_{\theta} \mathbb{E}_{t, \mathbf{X}_1, \mathbf{X}_0} \left\| \mathbf{u}_t^\theta(\mathbf{X}_t) - \frac{d}{dt} \mathbf{X}_t \right\|^2 \\ &= \arg \min_{\theta} \mathbb{E}_{t, \mathbf{X}_1, \mathbf{X}_0} \left\| \mathbf{u}_t^\theta(\mathbf{X}_t) - \frac{d\beta_t}{dt} \mathbf{X}_0 - \frac{d\alpha_t}{dt} \mathbf{X}_1 \right\|^2 \end{aligned}$$

Sampling from an FM model (cont.)

Algorithm 1 Sampling from an FM model with Euler method

- 1: Input: velocity field $\mathbf{u}_t^\theta$ (a DNN model), number of steps k
 - 2: Set $t = 0$
 - 3: Set stepsize $h = \frac{1}{k}$
 - 4: Draw a sample $\mathbf{x}_0 \sim p_{init} = \mathcal{N}(0, \mathbf{I})$
 - 5: **for** $i = 1, \dots, k$ **do**
 - 6: $\mathbf{x}_{t+h} = \mathbf{x}_t + h\mathbf{u}_t^\theta(\mathbf{x}_t)$
 - 7: Set $t \leftarrow t + h$
 - 8: **end for**
 - 9: return $\mathbf{x}_{t=1}$
-

Remark 1: In principle, any ODE solver can be applied to perform sampling for an FM model.

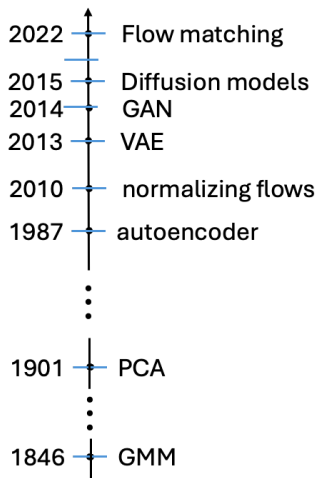
Images generated by flow matching models



Image from Flux, a particular flow matching model

Diffusion models (Lect. 14-16)

Diffusion Models was invented in 2015.



Prior Knowledge

- Let PDF of $\mathbf{X}_0$ be $p_{init} = \mathcal{N}(\mathbf{0}, \mathbf{I})$ (i.e., $p_{\mathbf{X}_0}(\mathbf{x}_0) = p_{init}(\mathbf{x})$)
- Let PDF of $\mathbf{X}_1$ be $p_{data}(\mathbf{x})$ (i.e., $p_{\mathbf{X}_1}(\mathbf{x}_1) = p_{data}(\mathbf{x})$)
- Define $\mathbf{X}_t = \beta_t \mathbf{X}_0 + \alpha_t \mathbf{X}_1$ where α_t and β_t satisfy:
 - α_t and β_t are scalar differentiable functions over $t \in [0, 1]$.
 - When $t = 0$, $\alpha_{t=0} = 0$ and $\beta_{t=0} = 1$.
 - When $t = 1$, $\alpha_{t=1} = 1$ and $\beta_{t=1} = 0$.
- We can obtain two equivalent ODEs

$$\boxed{\mathbf{u}_t^{\text{target}}(\mathbf{X}_t) = \frac{d\mathbf{X}_t}{dt} = \frac{d\beta_t}{dt} \mathbf{X}_0 + \frac{d\alpha_t}{dt} \mathbf{X}_1} \quad [ODE] \quad (4)$$

$\Updownarrow$

$$\boxed{\frac{d\mathbf{X}_t}{dt} = \left(\beta_t^2 \frac{d \log \alpha_t}{dt} - \frac{1}{2} \frac{d\beta_t^2}{dt} \right) \nabla_{\mathbf{X}_t} \log p_{t|1}(\mathbf{X}_t | \mathbf{X}_1) + \frac{d \log \alpha_t}{dt} \mathbf{X}_t} \quad [ODE] \quad (5)$$

where $p_{t|1}(\mathbf{X}_{t|1} | \mathbf{X}_1)$ is the conditional probability distribution of $\mathbf{X}_t$ given $\mathbf{X}_1$

Prior Knowledge (Lect. 14-16)

We show ODE (5) is equivalent to ODE (4):

1. The conditional distribution of $\mathbf{X}_t$ given $\mathbf{X}_1$ is a Gaussian distribution:

$$p_{t|1}(\mathbf{x}_t|\tilde{\mathbf{x}}) = \mathcal{N}(\alpha_t\tilde{\mathbf{x}}, \beta_t^2\mathbf{I}_d) \quad \text{where } \tilde{\mathbf{x}} \text{ drawn from } p_{data}(\cdot)$$

2. The **score function** $\nabla \log p_{t|1}(\mathbf{x}_t|\tilde{\mathbf{x}})$ takes the form of

$$\boxed{\nabla_{\mathbf{x}_t} \log p_{t|1}(\mathbf{x}_t|\tilde{\mathbf{x}}) = -\frac{\mathbf{x}_t - \alpha_t\tilde{\mathbf{x}}}{\beta_t^2} = -\frac{\boldsymbol{\epsilon}}{\beta_t^2}} \quad (6)$$

- Derivation of (6):

$$\begin{aligned} & \nabla_{\mathbf{x}_t} \log p_{t|1}(\mathbf{x}_t|\tilde{\mathbf{x}}) \\ &= \nabla_{\mathbf{x}_t} \log \left(\frac{1}{(2\pi)^{\frac{d}{2}} \beta_t^d} e^{-\frac{1}{2\beta_t^2} \|\mathbf{x}_t - \alpha_t\tilde{\mathbf{x}}\|^2} \right) \\ &= -\nabla_{\mathbf{x}_t} \left(\frac{1}{2\beta_t^2} \|\mathbf{x}_t - \alpha_t\tilde{\mathbf{x}}\|^2 + \text{constant} \right) \\ &= -\frac{\mathbf{x}_t - \alpha_t\tilde{\mathbf{x}}}{\beta_t^2} = -\frac{\boldsymbol{\epsilon}}{\beta_t^2} \end{aligned}$$

Objective of a diffusion model (Lect. 14-16)

Alternative objective function:

- **Input:** data samples $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^n$.
- **Assumption:** Assume $\{\mathbf{x}_i\}_{i=1}^n$ are drawn from the data distribution $p_{data}(\cdot)$.
- We train a neural network $\hat{\epsilon}_t^\theta(\mathbf{X}_t)$ over $t \in [0, 1]$ from the dataset $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^n$ by regression with respect to $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- Objective function for $\hat{\epsilon}_t^\theta(\cdot)$:

$$\begin{aligned} & \arg \min_{\theta} \mathbb{E}_{t \sim \mathcal{U}[0,1], \mathbf{x} \sim p_{data}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)} \left[\left\| \hat{\epsilon}_t^\theta(\mathbf{x}_t) - \epsilon \right\|^2 \right] \\ &= \arg \min_{\theta} \mathbb{E}_{t \sim \mathcal{U}[0,1], \mathbf{x} \sim p_{data}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)} \left[\left\| \hat{\epsilon}_t^\theta(\alpha_t \mathbf{x} + \beta_t \epsilon) - \epsilon \right\|^2 \right] \end{aligned} \quad (7)$$

- The associated ODE in terms of $\hat{\epsilon}_t^\theta(\cdot)$ is:

$$\begin{aligned} & \boxed{\frac{d\mathbf{x}_t}{dt} = \left(\beta_t^2 \frac{d \log \alpha_t}{dt} - \frac{1}{2} \frac{d \beta_t^2}{dt} \right) \frac{-(\mathbf{x}_t - \alpha_t \tilde{\mathbf{x}})}{\beta_t^2} + \frac{d \log \alpha_t}{dt} \mathbf{x}_t \quad \text{where} \quad \tilde{\mathbf{x}} \sim p_{data}} \\ & \approx \boxed{d\mathbf{x}_t = \left(\beta_t^2 \frac{d \log \alpha_t}{dt} - \frac{1}{2} \frac{d \beta_t^2}{dt} \right) \frac{-\hat{\epsilon}_t^\theta(\mathbf{x}_t)}{\beta_t} + \frac{d \log \alpha_t}{d} t \mathbf{x}_t} \quad \text{where} \quad \mathbf{x}_0 \sim p_{init} \end{aligned}$$

Sampling from a diffusion model (Lect. 14-16)

Algorithm 2 Sampling from a diffusion model

1: Input: neural network $\hat{\epsilon}_t^\theta$, number of steps k

2: Set $t = 0$

3: Set stepsize $h = \frac{1}{k}$

4: Draw a sample $\mathbf{x}_0 \sim p_{init} = \mathcal{N}(0, \mathbf{I})$

5: **for** $i = 1, \dots, k$ **do**

6: $\mathbf{x}_{t+h} = \beta_{t+h} \hat{\epsilon}_t^\theta(\mathbf{x}_t) + \alpha_{t+h} \frac{(\mathbf{x}_t - \beta_t \hat{\epsilon}_t^\theta)}{\alpha_t}$

[DDIM sampling procedure]

7: Set $t \leftarrow t + h$

8: **end for**

9: return $\mathbf{x}_{t=1}$

Remark 1: In principle, any ODE solver can be applied to perform sampling for a diffusion model.

Images generated by diffusion models (Lect. 14-16)



pictures from "Improved Distribution Matching Distillation for Fast Image Synthesis"

Video Generation via Sora by OpenAI

- Progress in 2024
 - Generated fake video clips
 - No sound generation in the video clips.
- Progress in 2025
 - Generated fake video clips
 - Generated sound coupled with the generated video clips.

Video Generation via Veo3 by Google

Progress in 2025

- generated video clip with text prompt "Off-road rally"
- generated video clip with text prompt "Sailor and the sea"
- Generated sound coupled with the generated video clips.

Image Editing (Lect. 17-18)

- DDIM-based Image Editing
 - DDIM inversion computes a latent representation of a clean image by solving the ODE from $t = 0$.
 - DDIM sampling computes a modified clean image with the obtained latent representation by solving the ODE until $t = 0$.

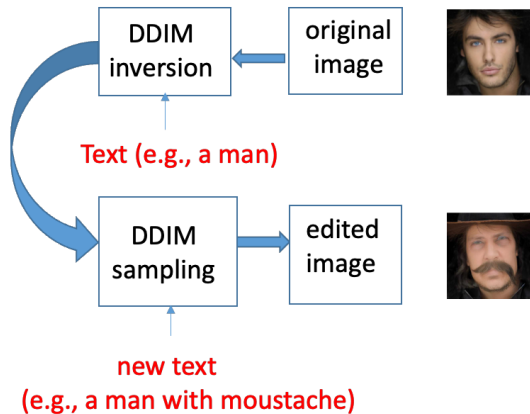


Image Editing (Lect. 17-18)

- BDIA-based Image Editing
 - The BDIA technique attempts to reduce the approximation error introduced DDIM-based image editing.

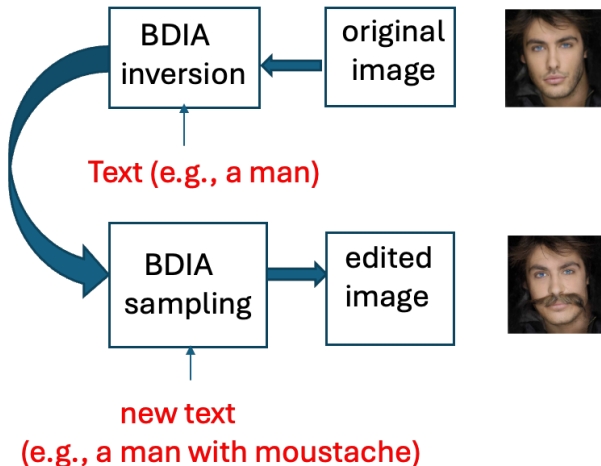


Image Editing (Lect. 17-18)

Input caption: “A man in glasses eating a doughnut in the park.”



Input Image



“... red-haired man...”



“glasses” → “sunglasses”



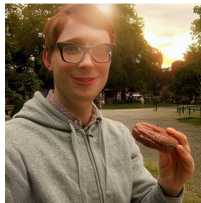
“angry man...”



“doughnut” → “pizza”



“glasses” → “Joker mask”



“...the park at sunset,”



“park” → “desert”

Machine Learning for Modeling Sequences (Lect. 19-20)

- When applying machine learning to sequences, we often want to turn an input sequence into an output sequence that lives in a different domain
 - Example 1: To turn a sequence of audio signals into a sequence of words.
 - Example 2: To turn a sentence in one language into a sequence in another language while preserving the meaning.
- When there is only a single sequence, we are interested in training a machine learning model to predict the next term in the sequence based the past terms.
 - In the training process, the target output sequence is the input sequence with an advance of 1 step.
 - Roughly speaking, ChatGPT is trained by predicting the next word in a document based on the previous words.
 - It reduces to unsupervised learning.

Modeling Sequences via Probability (Lect. 19-20)

- Suppose the joint distribution of the terms in sequences is characterized by $P_{data}(\{Y_i\}_{i=1}^n)$.
 - Assumption: Each Y_i is a discrete random variable, taking values from a discrete set Ω .
 - For example, for natural language processing, the discrete set includes all the possible words or subwords.
- It is known from probability theory that the joint distribution can be decomposed as

$$P_{data}(\{Y_i\}_{i=1}^n) = P_{data}(Y_1) \prod_{i=2}^n P_{data}(Y_i | \{Y_{k < i}\})$$

where $\{Y_{k < i}\}$ includes all the random variables up to index i .

Remark: For simplicity, we only consider discrete random variables. One can also extend the analysis to continuous case.

Auto-Regressive (AR) Models (Lect. 19-20)

- An AR model $P_{\theta}(\{Y_i\}_{i=1}^n)$ attempts to learn the conditional distribution $P_{data}(Y_i|\{Y_{k<i}\})$ for all index i .
- Once an AR model is trained, it can be used to generate new sequences in a recurrent manner, one term per timestep according to the learned conditional probability.
- In general, those popular AR models in the literature are mainly different in their architectures.

Popular AR models in the literature:

- Recurrent neural networks
- Transformer-based AR models
 - They dominate natural language processing (NLP)
 - Transformer can also be used in diffusion and flow-matching models. Generally speaking, Transformer is not an AR model in diffusion and flow-matching models because each token uses all other tokens for updating.
- State space models (variants of RNN models)

Questions?